

REPORTE: De los fundamentos de las redes neuronales a las LSTM

Abraham Jain Jiménez

Contents

1	El perceptrón	2
2	El perceptrón como la primera red neuronal	3
3	Funciones de activación	4
4	El perceptrón mult capas	4
5	Forwardpass	5
6	RNN	6
7	La memoria en las RNN: Long-Term Dependencies	7
8	Redes LSTM	8
9	Estructura de las LSTM	8

1 El perceptrón

El perceptrón simple es un modelo de aprendizaje supervisado, un clasificador lineal que da origen a las redes neuronales.

Se trata de un modelo convergente, pues A. Novikoff (*Novikoff, A. (1962). On convergence proofs on perceptrons. Proceedings of the Symposium on the Mathematical Theory of Automata, 12, 615–622*) demuestra que el perceptrón alcanza un error de entrenamiento cero si y solo si los datos son linealmente separables.

Los clasificadores lineales parten de una frontera de decisión definida mediante un hiperplano, en donde hablando de clasificación binaria, se asocia una regla de decisión.

Dado un conjunto de datos de entrenamiento $D = \{\vec{x}_k \in \mathbb{R}^d, t_k \in \{+1, -1\}\}_{k=1}^M$, en donde t_k es la etiqueta correspondiente al problema de aprendizaje supervisado (+1 para la clase positiva), -1 para la clase negativa), el hiperplano está definido por la ecuación

$$\vec{w}^T \vec{x} + b = 0 \quad (1)$$

En donde b es el sesgo (bias) y $\vec{w}$ es el vector de pesos, el cual es normal al hiperplano y va a caracterizarlo, por lo que la regla de decisión está dada mediante la función

$$f(\vec{x}) = \begin{cases} 1, & \text{si } \vec{w}^T \vec{x} + b > 0 \\ 0, & \text{en cualquier otro caso} \end{cases} \quad (2)$$

De aquí, podemos reescribir f , definiendo el perceptrón como la función

$$f(\vec{x}) = \theta(\vec{w}^T \vec{x} + b) \quad (3)$$

En donde θ es una función de activación que corresponde a la función escalón.

El perceptrón se entrena mediante el siguiente algoritmo:

1. Efectuamos múltiples ciclos sobre los M datos de entrenamiento ($k \in \{1, 2, \dots, M\}$):

- Si $\vec{x}_k$ está correctamente clasificado, se asigna su etiqueta: $f(\vec{x}_k) = t_k$
- Si $\vec{x}_k$ no está correctamente clasificado, se tiene que $f(\vec{x}_k) \neq t_k$, por lo que se actualiza el vector $\vec{w}$ mediante la regla:

$$\begin{cases} \vec{w} \rightarrow \vec{w} + \eta \vec{x}_k t_k \\ b \rightarrow b + \eta t_k \end{cases}$$

En donde η es un término de proporcionalidad conocido como ritmo de aprendizaje y t_k es la etiqueta real.

2. El algoritmo converge cuando cada $\vec{x}_k$ está correctamente clasificado.

Siendo que el perceptrón converge siempre para datos linealmente separables, hablamos de un problema de minimización de la forma

$$ArgMin_{\vec{w}, b} \{L(\vec{w}, b)\} \quad (4)$$

En donde la función de pérdida es:

$$L(\vec{w}, b) = \frac{1}{M} \sum_{k=1}^M \max\{0, -(\vec{w}^T \vec{x}_k + b)t_k\} \quad (5)$$

2 El perceptrón como la primera red neuronal

Con base en lo anterior, el perceptrón puede ser programado tomando como base dos puntos:

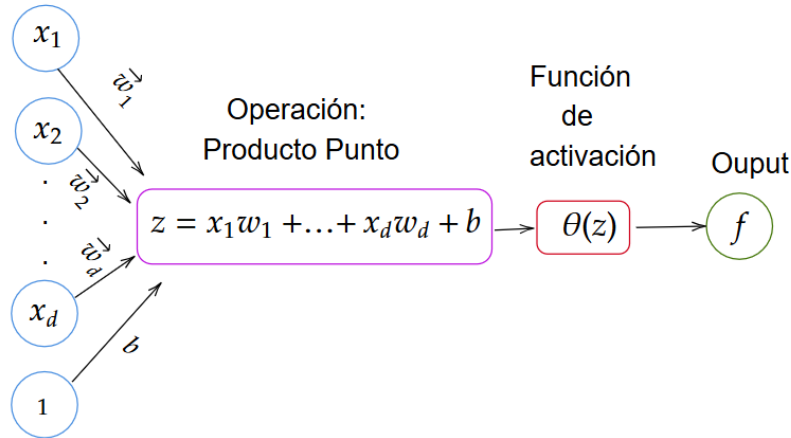
1. Calcular $z = \vec{w}^T \vec{x} + b$.
2. Aplicar la función de activación θ sobre z : $\theta(z) = f$

Reescribiendo los vectores $\vec{x}$ y $\vec{w}$ mediante su forma aumentada

$$\vec{x}' = \begin{pmatrix} \vec{x} \\ 1 \end{pmatrix}, \vec{w}' = \begin{pmatrix} \vec{w} \\ b \end{pmatrix}$$

podemos describir este procedimiento mediante una estructura de grafo:

Inputs



3 Funciones de activación

La función escalón como función de activación tiene una desventaja, pues presenta distontinuidad, y la derivada antes y después de la activación es la misma. Por ello, se han desarrollado otro tipo de funciones de activación, en donde las más comunes utilizadas en Deep Learning son:

- Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (6)$$

- Tanh:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad (7)$$

- ReLU:

$$\text{ReLU}(z) = \begin{cases} z, & \text{si } z > 0 \\ 0, & \text{en cualquier otro caso} \end{cases} \quad (8)$$

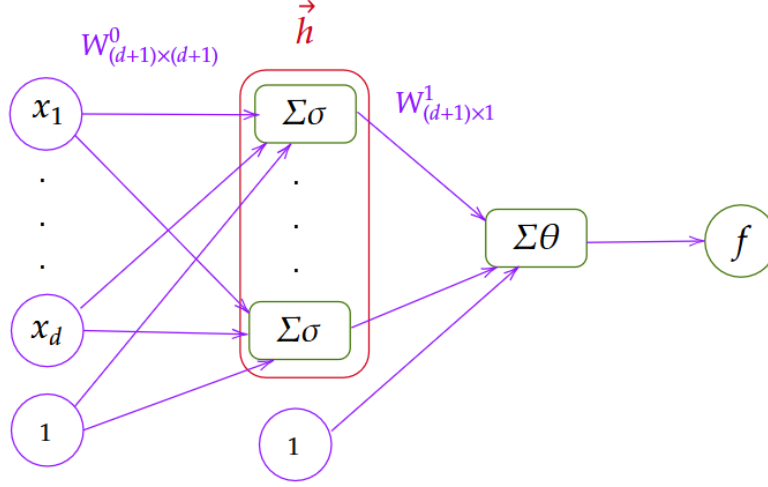
4 El perceptrón multicapas

El funcionamiento del perceptrón simple está condicionado a la separabilidad lineal de los datos, por lo que no modela relaciones no lineales. Sin embargo, la no linealidad se aborda al agregar capas ocultas (hidden layers) a la red neuronal que constituye el perceptrón. Este es el fundamento bajo el cual se construye el perceptrón multicapas (MLP).

En el MLP, a los nodos input que corresponden a los valores de las entradas del vector de características $\vec{x} \in \mathbb{R}^d$, se le asocian sus respectivos vectores de pesos $\vec{w} \in \mathbb{R}^d$, que concatenados pueden escribirse como una matriz de pesos.

Así mismo, cada capa oculta corresponde a un conjunto de valores arrojados por un grupo perceptrones, los cuales se asocian a las entradas de un vector $\vec{h} \in \mathbb{R}^{d+1}$ que va a caracterizar a esa capa oculta en donde se aborda la no linealidad.

Dada una función de activación asociada, σ , podemos generalizar la notación y esquematizar el sistema del perceptrón multicapas (en este caso con una sola capa) como sigue:



Esto es tomando nuevamente la convención de tomar los vectores aumentados $\vec{x}'$ y $\vec{w}'$.

Poniendo de ejemplo el caso en que tenemos solo una capa oculta con $\vec{x} \in \mathbb{R}^2$, entonces $\vec{x}' = \begin{pmatrix} x_1 \\ x_2 \\ 1 \end{pmatrix} \in \mathbb{R}^3$, por lo que tendríamos una matriz de pesos $W^0_{3 \times 3}$ como conexión con la capa oculta, un vector $\vec{h}^0$ describiéndola, y una nueva matriz de pesos $W^1_{3 \times 1}$. Finalmente, el modelo clasificador no lineal se escribiría como

$$f(\vec{x}) = \sigma(W^1 \vec{h}^0 [W^0 + b^0] + b^1) \quad (9)$$

5 Forwardpass

Los ejemplos anteriores se han ilustrado para una sola capa oculta, sin embargo, pueden formularse redes neuronales con N capas ocultas. A este número de hidden layers se le conoce como la profundidad de la red.

De lo anterior, dada una red neuronal (en nuestro caso el MLP) con N capas ocultas (comunmente la notación comienza desde el cero), tiene asociado al conjunto de vectores $\{\vec{h}^i\}_{i=0}^N$ y al conjunto de matrices de pesos $\{W^j\}_{j=0}^{N+1}$. Así, definimos el proceso forwardpass como la evaluación de todos los elementos de la red neuronal ($f(\vec{x})$) de izquierda a derecha.

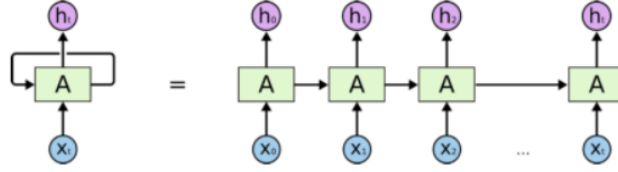
Por ejemplo, para un MLP con $\vec{x} \in \mathbb{R}^2$, una función de activación σ y dos capas ocultas, se asocian las matrices $\{W^0, W^1, W^2\}$ y los vectores $\{\vec{h}^0, \vec{h}^1\}$. Por lo que el modelo quedaría como:

$$f(\vec{x}) = \sigma[W^2\vec{h}^1(\sigma(W^1\vec{h}^0(W^0\vec{x} + b^0) + b^1) + b^2] \quad (10)$$

6 RNN

Las redes neuronales recurrentes (RNN) son modelos de aprendizaje supervisado especializados en trabajar con secuencias de valores, como el análisis de series de tiempo o, en general, de cualquier señal en el tiempo. Al trabajar con secuencias de valores, las RNN poseen una estructura descrita mediante ciclos.

A diferencia del MLP que procesa un vector fijo las RNN procesan una secuencia de vectores. Consideremos una RNN, A , con un conjunto de vectores input $\{\vec{x}_t \in \mathbb{R}^d\}_{t=0}^T$ tal que cada $\vec{x}_t$ arroja un vector output $\vec{h}_t \in \mathbb{R}^p$ (la dimensión p es un hiperparámetro que regula la complejidad del sistema) y su respectivo vector de etiqueta asociado al problema de aprendizaje supervisado $\vec{y}_t \in \mathbb{R}^d$, por lo que la estructura de una RNN puede ser esquematizada como:



Entonces, los ciclos de las RNN operando en cada paso de tiempo t , a partir de un vector de características $\vec{x}_t$, son de la forma

$$\vec{x}_t \rightarrow \vec{h}_t \rightarrow \vec{y}_t \quad (11)$$

Análogamente al MLP, los vectores $\vec{h}_t$ son estados ocultos, que ahora de acuerdo a la estructura de las RNN, dependen del pasado, por lo que el modelo en cada paso de tiempo t puede ser descrito analíticamente mediante:

$$\vec{h}_t = \sigma(\vec{h}_{t-1}, \vec{x}_t) \quad (12)$$

La función de activación típica de las RNN es $\sigma = \tanh$, por lo que tenemos que estas pueden ser descritas mediante las ecuaciones

$$\vec{h}_t = \tanh(W_{\vec{x}}\vec{x}_t + W_{\vec{h}}\vec{h}_{t-1}) \quad (13)$$

$$\vec{y}_t = W_{\vec{y}}\vec{h}_t \quad (14)$$

En donde $W_{\vec{x}}$ es una matriz de pesos de orden $p \times d$ llamada input-hidden, $W_{\vec{h}}$ es de orden $p \times p$ llamada hidden-hidden y $W_{\vec{y}}$ es de orden $d \times p$ llamada hidden-output.

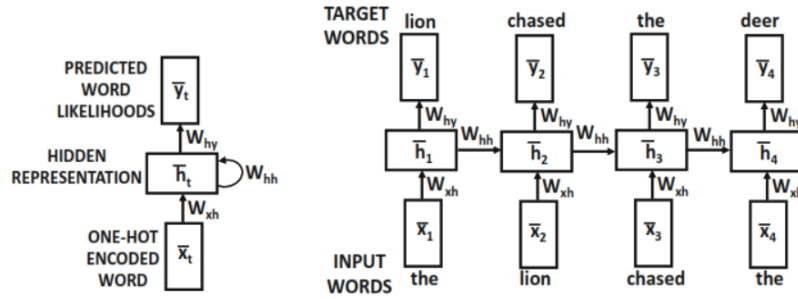


Figure 1: Ejemplo de RNN: Aggarwal, C. C. (2023). Neural networks and deep learning: A textbook (2nd ed.). Springer.

7 La memoria en las RNN: Long-Term Dependencies

De acuerdo a lo expuesto anteriormente, vemos que las RNN pueden aprender utilizando la información del pasado, pues las capas ocultas asociadas a los vectores $\vec{h}$ introducen una memoria dentro de este tipo de redes.

Sin embargo, en la práctica, conforme la brecha temporal entre los datapoints de interés va creciendo, el modelo no es capaz de conectar la información, por lo que no aprende.

A este problema se le conoce como Long-Term Dependencies, y es solucionado mediante un tipo especial de RNN conocido como LSTM.

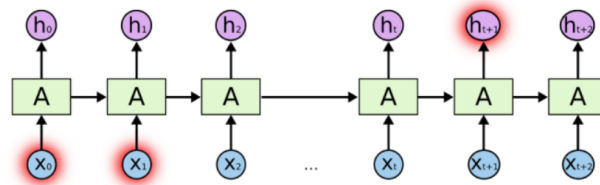


Figure 2: Ejemplo del problema

8 Redes LSTM

Las redes LSTM (Long Short Term Memory) son un tipo especial de RNN diseñadas para abordar el problema long-term dependency. Poseen una estructura interior distinta a las RNN típicas, pues en lugar de neuronas ocultas, introducen sistemas llamados células, que juegan el mismo rol pero con un comportamiento más avanzado.

La estructura de este tipo de redes se fundamenta en los siguientes componentes:

- Input: $\vec{x}_t$
- Input Gate: $\vec{i}_t$
- Forget Gate: $\vec{f}_t$
- Output Gate: $\vec{h}_t$
- Cell State: $\vec{C}_t$
- Output: $\vec{h}_t$

El concepto clave de las LSTM es el estado de la célula (cell state), el cual hace referencia a la memoria interna de largo plazo en la red, pues se puede remover o añadir información a este estado regulando la información mediante compuertas.

9 Estructura de las LSTM

Dado un conjunto de vectores input $\{\vec{x}_t \in \mathbb{R}^d\}_{t=0}^T$, podemos describir cómo opera la estructura de las redes LSTM sobre un $\vec{x}_t$ paso por paso como sigue:

1. Decidir qué información desechar:

Esta decisión se hace mediante una función de activación sigmoide (σ) llamada compuerta de olvido (Forget Gate).

En cada paso de tiempo, recibe los vectores $\vec{h}_{t-1}$ y $\{\vec{x}_t$ y arroja un valor dentro del intervalo $[0, 1]$ que hace referencia a qué tanto se va a conservar esa información. El 1 representa que la información se conserva por completo, el 0 que se olvida por completo.

La forget gate en cada paso de tiempo se describe analíticamente mediante

$$\vec{f}_t = \sigma(W_f \cdot [\vec{h}_{t-1}, \vec{x}_t] + \vec{b}_f) \quad (15)$$

En donde tenemos que $\vec{f}_t \in \mathbb{R}^p$, cuyas componentes son de la forma:

$$\{f_t^{(k)}\} \in [0, 1]^p_{k=0} \quad (16)$$

Pues se aplica la función σ sobre cada componente del vector $W_f \cdot [\vec{h}_{t-1}, \vec{x}_t] + \vec{b}_f$.

2. Decidir qué información nueva se va a almacenar en la memoria (en el cell state):

En este paso se definen dos procesos. El primero es que la compuerta de entrada (input gate) decide qué valores se van a actualizar mediante la función sigmoide. De manera análoga al primer paso, esto se describe analíticamente mediante:

$$\vec{i}_t = \sigma(W_i \cdot [\vec{h}_{t-1}, \vec{x}_t] + \vec{b}_i) \quad (17)$$

Tal que $\vec{i}_t \in \mathbb{R}^p$ y $\{i_t^{(k)}\} \in [0, 1]^p_{k=0}$.

El segundo proceso consiste en que una función de activación $\tanh$ nos va a permitir crear un nuevo vector, $\vec{C}'$, que pueda ser añadido al cell state:

$$\vec{C}'_t = \tanh(W_C \cdot [\vec{h}_{t-1}, \vec{x}_t] + \vec{b}_C) \quad (18)$$

Tal que $\vec{C}'_t \in \mathbb{R}^p$ y $\{C_t'^{(k)}\} \in [-1, 1]^p_{k=0}$.

3. Actualizar el cell state antiguo:

Actualizamos el estado antiguo $\vec{C}'_{t-1}$ por un nuevo estado $\vec{C}_t$, decidido por los pasos anteriores, mediante:

$$\vec{C}_t = \vec{f}_t * \vec{C}'_{t-1} + \vec{i}_t * \vec{C}'_t \quad (19)$$

4. La output:

Con base a los pasos anteriores, decidimos qué es lo que arrojará la salida de la red, mediante las ecuaciones:

$$\vec{o}_t = \sigma(W_o \cdot [\vec{h}_{t-1}, \vec{x}_t] + \vec{b}_o) \quad (20)$$

$$\vec{h}_t = \vec{o}_t * \tanh(\vec{C}_t) \quad (21)$$

En donde $\vec{o}_t \in \mathbb{R}^p$ y $\{o_t^{(k)}\} \in [0, 1]^p_{k=0}$ y $\vec{h}_t \in \mathbb{R}^p$ y $\{h_t^{(k)}\} \in [-1, 1]^p_{k=0}$